

Generative policies

EN.601.[4|6]98 Hands-on Robot Learning

Krishna Murthy Jatavallabhula

Today's focus: Generative (mostly, single-task) policies

- Last *lecture*: behavioral cloning, action chunking, ACT, why policy evaluation is difficult
- Today:
 - multimodality, generative policies that capture multimodality, how to train/steer/adapt them
 - diffusion, flow matching (generative modeling in modern ML)
 - diffusion policies ✨
 - how to control diffusion policies (steering, composing, inference-tweaks)

By the end of the lecture, you'll be able to

- Appreciate the challenges of learning multimodal action distributions via simpler L1/L2 objectives (as we did in previous lectures)
- Understand how diffusion models work at training and inference time
- Explain what changes are needed to enable diffusion models to generate action chunks
- Develop an intuitive understanding of steering and composing policies
- Develop a deeper understanding/appreciation of the policy inference stack and the various methods in play today when deploying trained (generative) policies

Administrivia

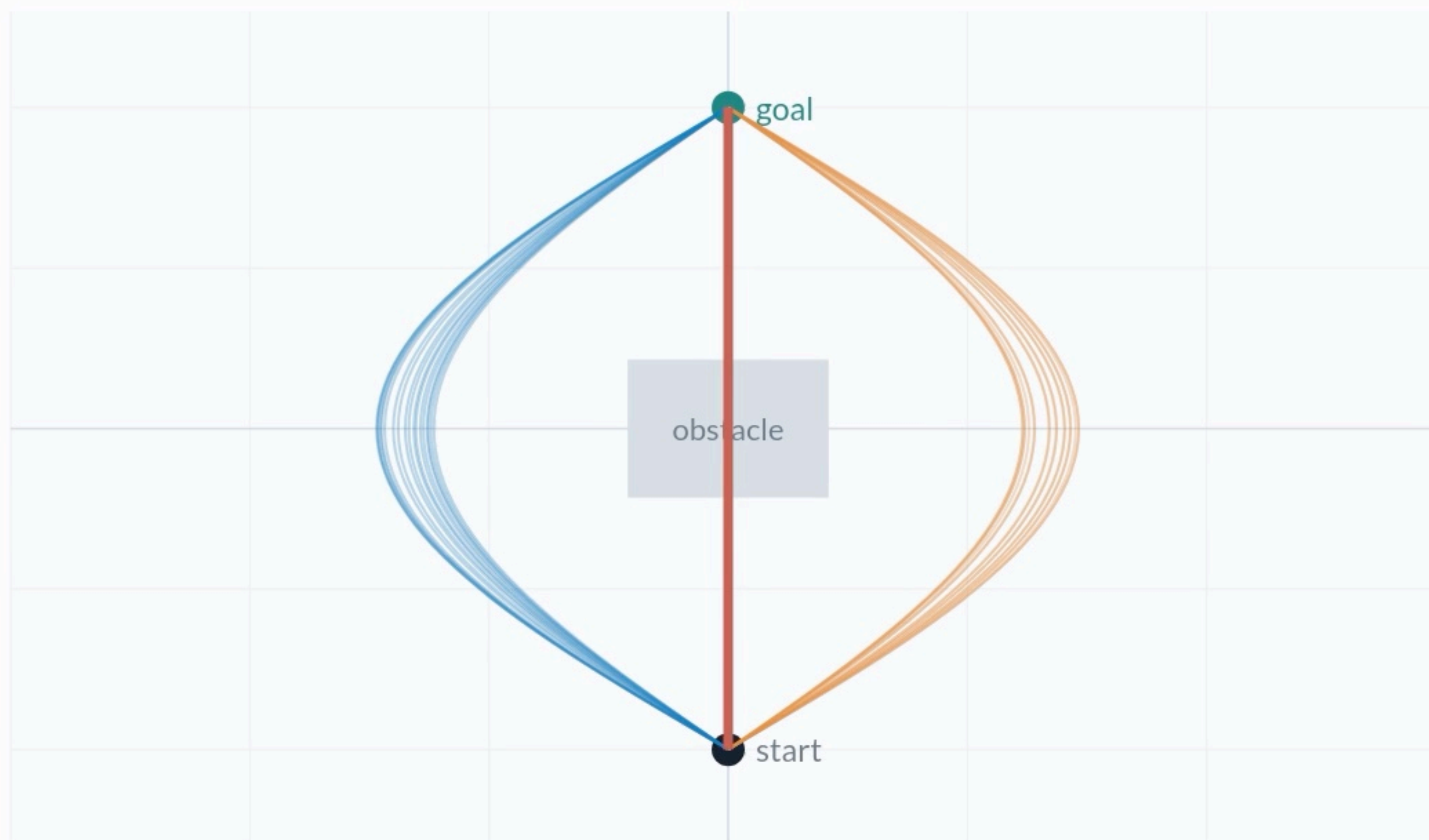
- Assignment 0 is up on canvas
- Most of you have done this already (last week's assembly session!)
- (Soft deadline) Friday, October 9
- [Hard deadline] Monday, October 12
- Robot use, care, and troubleshooting
- Office hours:
 - Xinyi Yin - Wednesdays (2-3 pm), Shriver 001
 - Yinzhe (Steven) Zhou - Fridays (9.30-10.30 am), 265W in Wyman Park building
 - Aleks Santari - Tuesdays (6-7 pm), Malone 122
 - Krishna (i.e., me) - Thursdays (12-1 pm), Hackerman 226E

Multimodality in robot learning

- Often, multiple ways of accomplishing the same objective
- E.g., robot encounters an obstacle is on the path to an object. Can go left or right. But observations do not tell us what path a demonstrator would take (i.e., non-Markovian state)
- If a policy is trained with a mean-squared error objective, it predicts an ‘average’ path that goes ‘through’ the obstacle, which is undesirable
- Avg. is a good guess if there’s a strong central path and other paths are small variations to the central path. It is a poor guess if the paths are distinctly separated into ‘modes’
- Generative models (in modern ML parlance) represent multimodal distributions over high-dimensional variables (such as action chunks)

What should the policy predict?

Two demonstrated strategies



Conditional mean

A coherent sample

PREDICTING AN AVERAGE

Both routes are valid. Their average can pass directly through the obstacle.

$$\operatorname{argmin} E[\|a - \hat{a}\|^2 | o] = E[a | o]$$

A sampled behavior can choose a side and stay with it. Random jitter around the mean does not recover this structure.

Left-route frequency

50%

Desirable characteristics of a generative policy

Four desirable characteristics (as of Q3 2026)

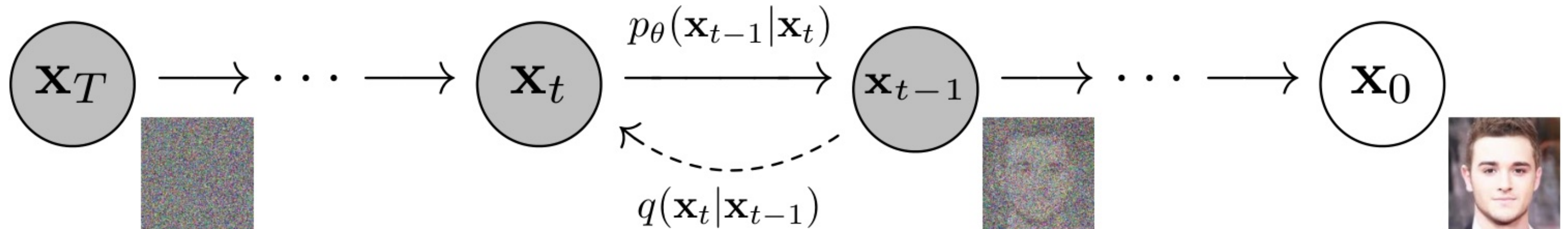
- Plausibility: predicted action sequence must be plausible (i.e., no averaging across disjoint modes)
- Multimodality: allow multiple good solutions — each sampled action chunk is inherently consistent
- Conditioning: prediction must be contingent on what the robot observes
- Configurability: enable choosing across multiple *samples*, based on preferences, and the ability to update the predicted actions based on custom criteria

Commonly used action generation methods

Approach for action generation	What the model predicts	How inference works
Feedforward neural network (deterministic, point-estimate)	Action chunk	One forward pass
Mixture / latent model	Mode probabilities /a latent-conditioned output vector	Choose or sample a mode / latent
Implicit / energy-based model	<i>Energy</i> over candidate actions	Search/optimize for low-energy candidates
Diffusion / flow	A local <i>refinement</i> to the previous iterate (applied multiple times to get actions)	Transform (denoise) a noise vector to an action chunk

Diffusion models

Forward and Reverse diffusion processes

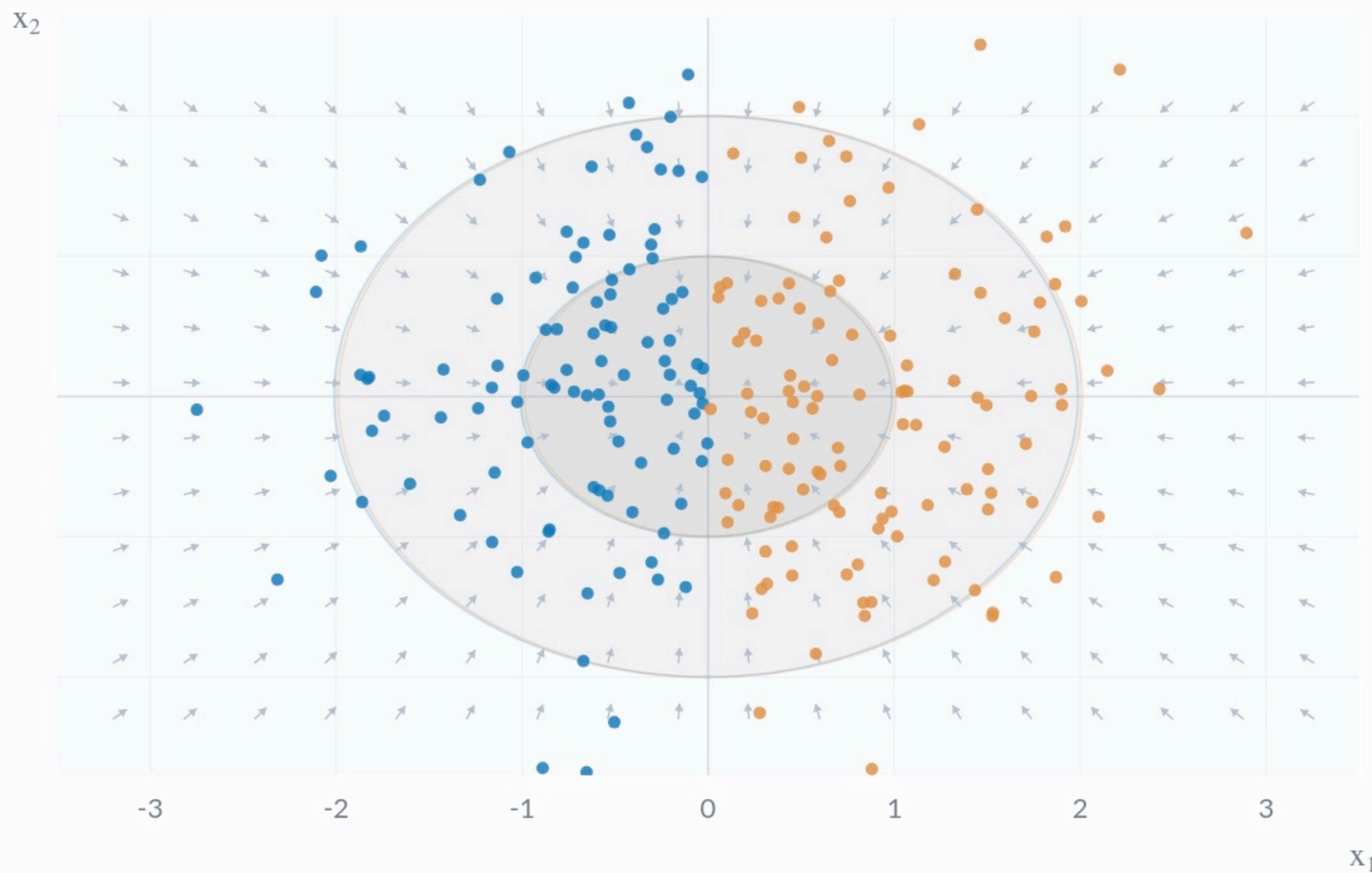


- Forward process: progressively corrupt data with a *known* noise process
- Reverse process: *learn* transitions that turn noise back into plausible samples

Noising and De-noising

- Iteratively add noise to a data sample: $x^k = \sqrt{\alpha_k} x^{k-1} + \sqrt{1 - \alpha_k} \epsilon$
- Noise sample at iteration k: $x^k = \sqrt{\bar{\alpha}_k} x^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon$ where $\bar{\alpha}_k = \prod_{i=1}^k \alpha_i$
- We can directly draw a noise sample at iteration k in one-shot, w/o needing to sequentially add noise from iteration 0 through k
- The first term preserves (some part of) the original signal. The second term adds (Gaussian) noise
- Denoiser learns what is *plausible* under the data distribution
- Once trained, we do not need to know the noise pattern (moreover, we do not / cannot know the noise pattern at inference time)

Can we turn noise into a useful sample?



A KNOWN TOY DENOISER

Repeated local updates transform a Gaussian sample cloud into a multimodal distribution.

$$\hat{x}_0 = (x_t - \sigma_t \hat{\epsilon}) / \alpha_t$$

The denoiser is exact for this Gaussian-mixture example. The deterministic DDIM sampler still starts from a random initial sample.

Noise level t

0.99

Sampling step

0 / 40

Training a diffusion model

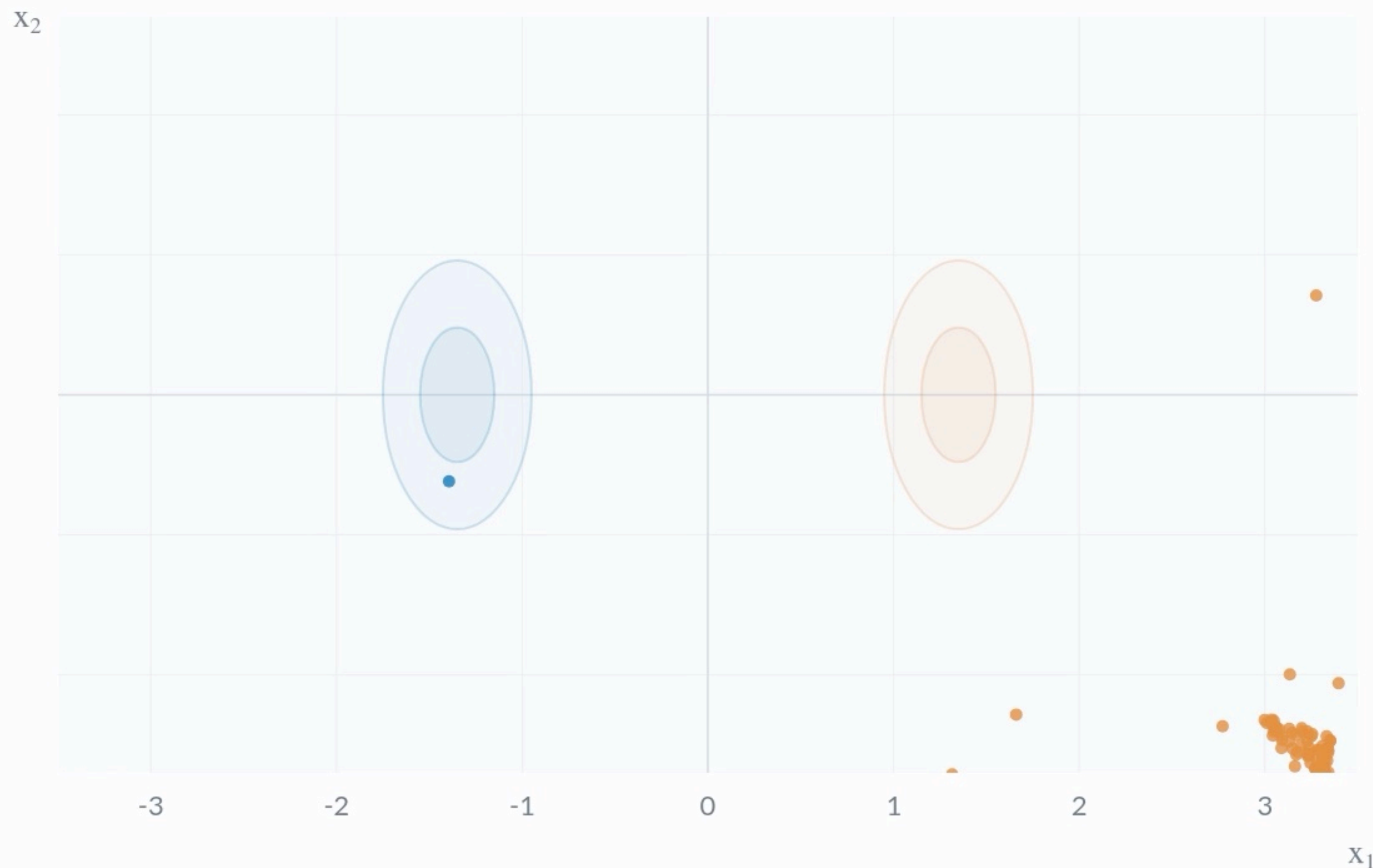
- Sample a datapoint x^0 and a *timestep* k uniformly $k \sim \text{Uniform}(1 \cdots T)$
- Sample Gaussian noise vector using $\epsilon \sim \mathcal{N}(0, I)$
- Construct x^k and train the diffusion model to predict the noise that was added

$$\mathcal{L} = \|\epsilon - \epsilon_{\theta}(x^k, k)\|^2$$

- Trained via a regression (e.g., mean squared-error) objective
- In this case, MSE is not bad (why?)
- Network takes timestep (k) as input (i.e., do not need to train a different network for each value of k)
- Additional (conditioning) inputs possible (and often necessary)

What does the network actually learn?

Generated samples from this checkpoint



ACTUAL TRAINING CHECKPOINT

A small neural network learns to predict noise from a noisy point, its noise level, and a condition.

$$L = E[\|\varepsilon - \varepsilon\theta(x_t, t, c)\|^2]$$

Each optimizer update samples a fresh noise level. Sampling uses the trained model repeatedly. Fixed initial samples make checkpoint comparisons fair.

Optimizer updates

0

Network

8-64-64-2

Inference with diffusion models

- Sample a noise vector (usually from a standard normal distribution)
- Start with $k = 0$, and gradually denoise, using the same noise schedule as the model is trained to expect, and denoise for T timesteps
- Implementation tricks/notes:
 - Network predicts noise at each step. We add it to the previous iterate to eventually end up with a clean datapoint (if model is well-trained)
 - Timestep (k) is often encoded sinusoidally (positional encoding/embedding)
$$\text{emb}(k)_{2i} = \sin(k \cdot 10000^{-2i/d}), \quad \text{emb}(k)_{2i+1} = \cos(k \cdot 10000^{-2i/d}), \quad i = 0, \dots, \frac{d}{2} - 1$$
 - Reverse step samplers vary: DDPM adds *fresh* noise each denoising step; DDIM doesn't

Diffusion models: summary

- Forward process — iteratively adds noise to a clean datapoint (usually, until the distribution at timestep T is standard normal)
- Reverse process (learned) — de-noises a noise vector (usually sampled from a standard normal distribution) until we get a plausible datapoint
- Can be trained using MSE loss (w/ careful attention to batch selection, variance schedules, and timestep encoding)
- Can condition on additional variables (not just noise vector and timestep)
- Caveat: always need to begin with a standard distribution (such as std normal), and can only have a target timestep (k) as a constraint, not a source timestep (always presumed 0 or $k-1$ depending on the sampler)
- Fix: flow-matching!

Diffusion policy

Our first ‘generative’ policy

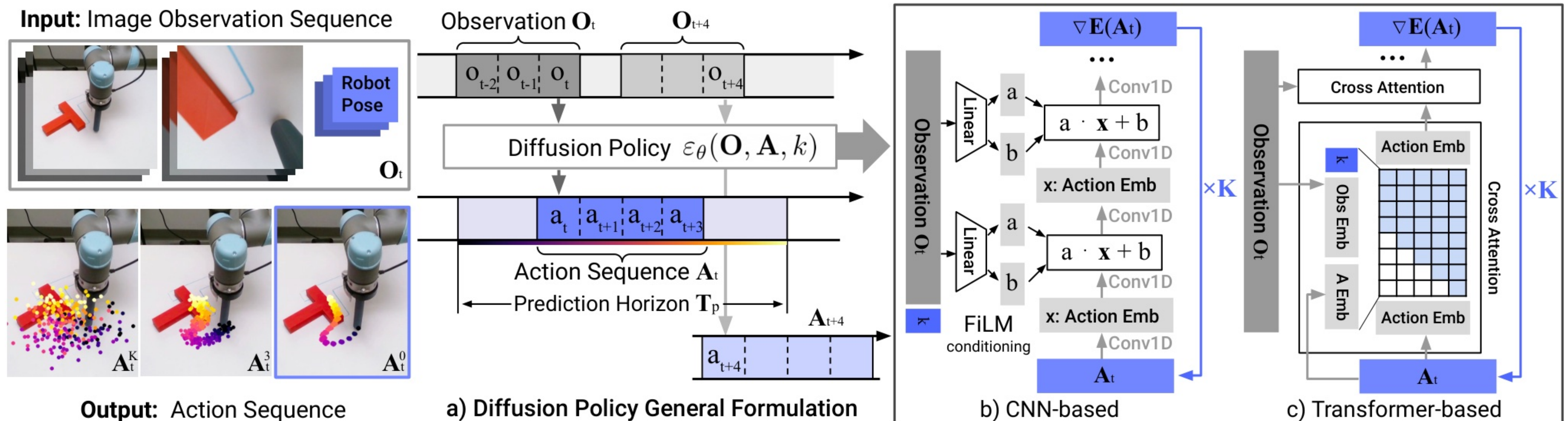
Diffusion policy: Action prediction using a diffusion process

- Given input observations (images, robot state), predict actions (action chunks)
- E.g., for SO-101, predict 5 joint positions (or change in positions) and gripper command (common choices: absolute gripper width, change in gripper state, binary open/close command)
- Action chunk size is typically 0.5-1 second's worth of actions (for SO-101 if using LeRobot, the default is closer to 0.5 sec)
- In diffusion policy, actions are predicted via a diffusion process, based on trajectories collected from demonstrations

$$A_t = [a_t, a_{t+1}, \dots, a_{t+H_p-1}] \in \mathbb{R}^{H_p \times d_a}$$

Diffusion policy: model architecture

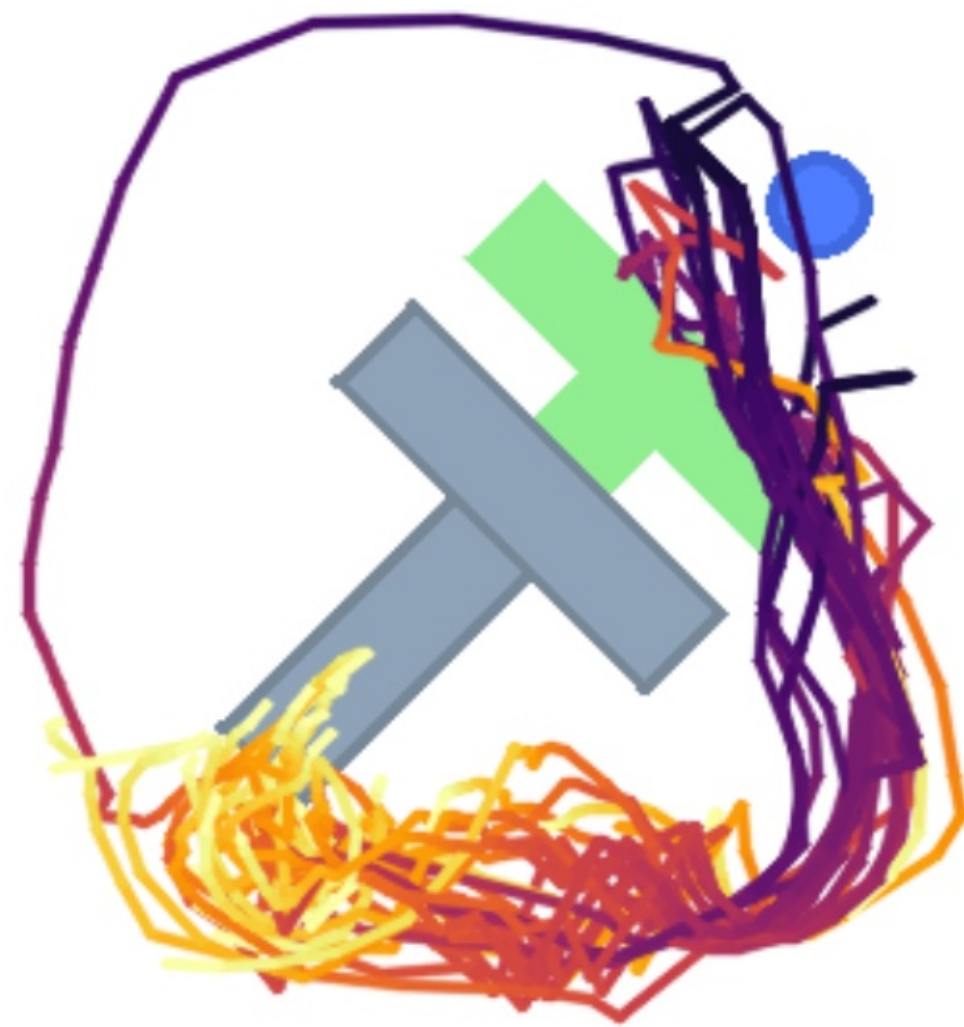
- Two components of the model: vision / observation encoder, action head
- Vision encoder (CNN / Transformer) computes visual features
- Diffusion model (a.k.a. action head / action expert) takes visual features, a denoising timestep, and a noise vector and denoises to obtain action chunks



Diffusion policy: qualitative comparison wrt baselines



Diffusion Policy



LSTM-GMM



BET



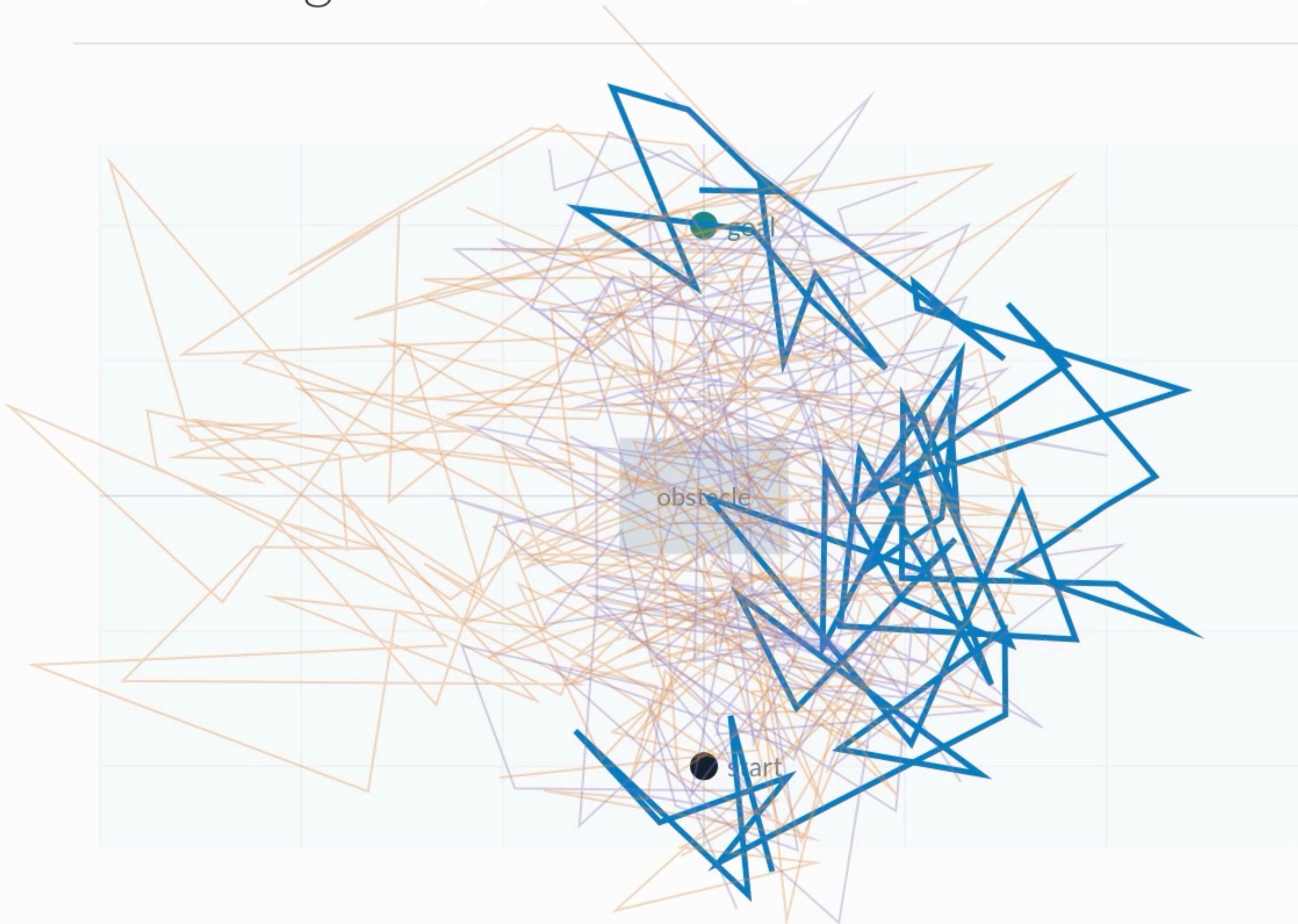
IBC

Diffusion policy: data preprocessing, training objective

- Assume we're provided a dataset of demonstrations (images, robot poses, actions)
- Dataset is preprocessed into a collection of action chunks
- Exercise care to ensure that
 - Stationary frames (zero actions) at the start of an episode are removed (else trained policies find it hard to initiate movement)
 - A small number of frames are captured after the end of the demonstration (or zero-pad if trajectory ends immediately on task completion)
 - Gripper actions and other joint actions are appropriately normalized and unnormalized
- Diffusion policy is trained using a denoising objective (predict noise at timestep k conditioned on sampled noise, timestep encoding, and visual encoder output embedding)

$$A_t^k = \sqrt{\bar{\alpha}_k} A_t + \sqrt{(1 - \bar{\alpha}_k)} \epsilon \quad L = E[\|\epsilon - \epsilon_\theta(A_t^k, O_t, k)\|^2]$$

Denoising an entire action chunk



A SEQUENCE OF COMMANDS

Refine the whole future chunk, then execute only a prefix before observing again.

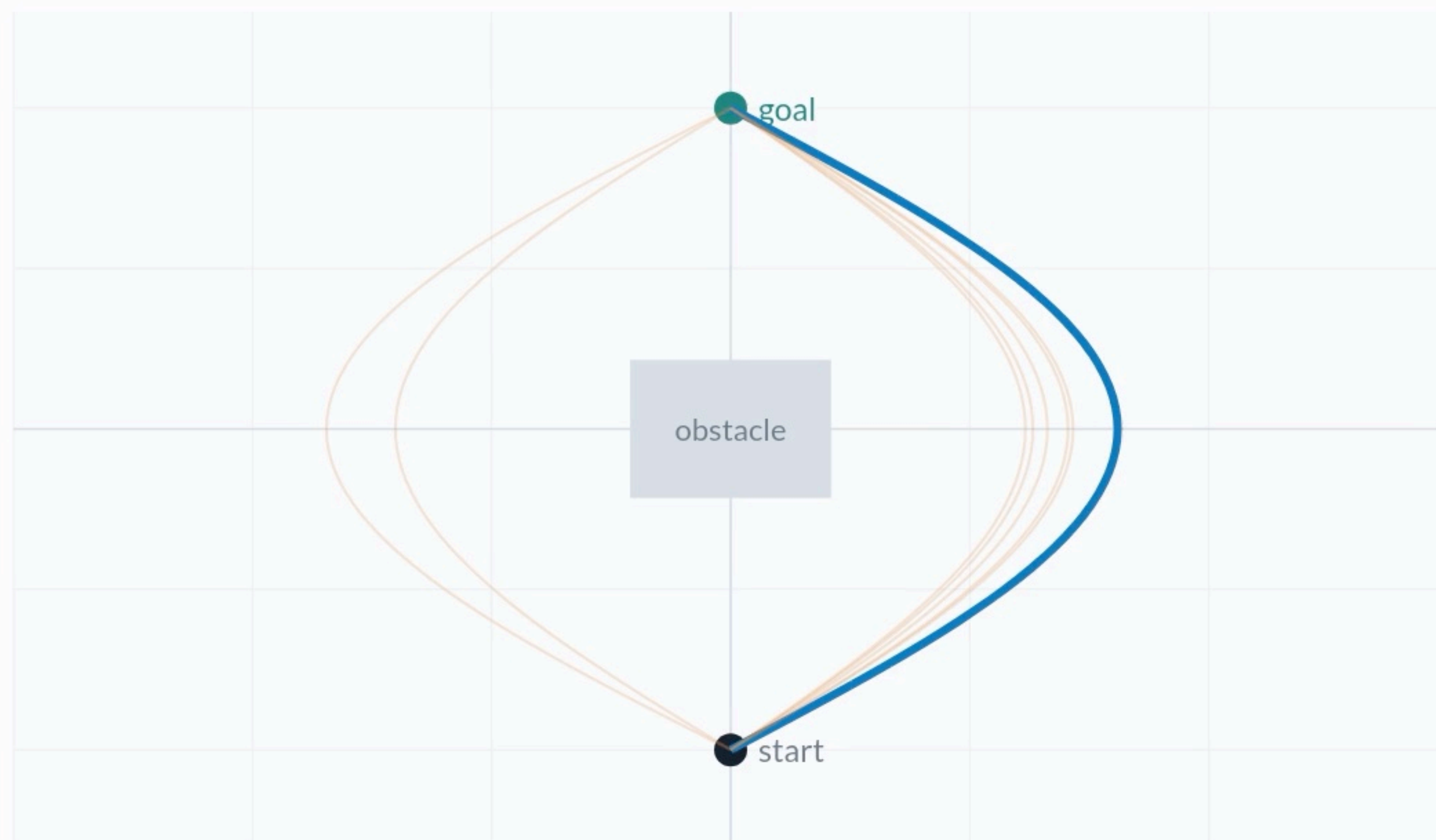
$$A \in \mathbb{R}^{(H_p \times d_a)}$$

This toy decodes a learned waypoint into a path. A full Diffusion Policy directly denoises every action coordinate. The noisy paths are never sent to a robot.

Denoising complete

0%

Same noise, different observations



HOLD THE INITIAL NOISE FIXED

The condition changes the output even when every sampling call starts from the same noise.

$$A = F\theta(O, z)$$

Here c is a scalar toy condition. In a visuomotor policy, O contains image features and proprioception. Keeping z fixed does not freeze the action trajectory.

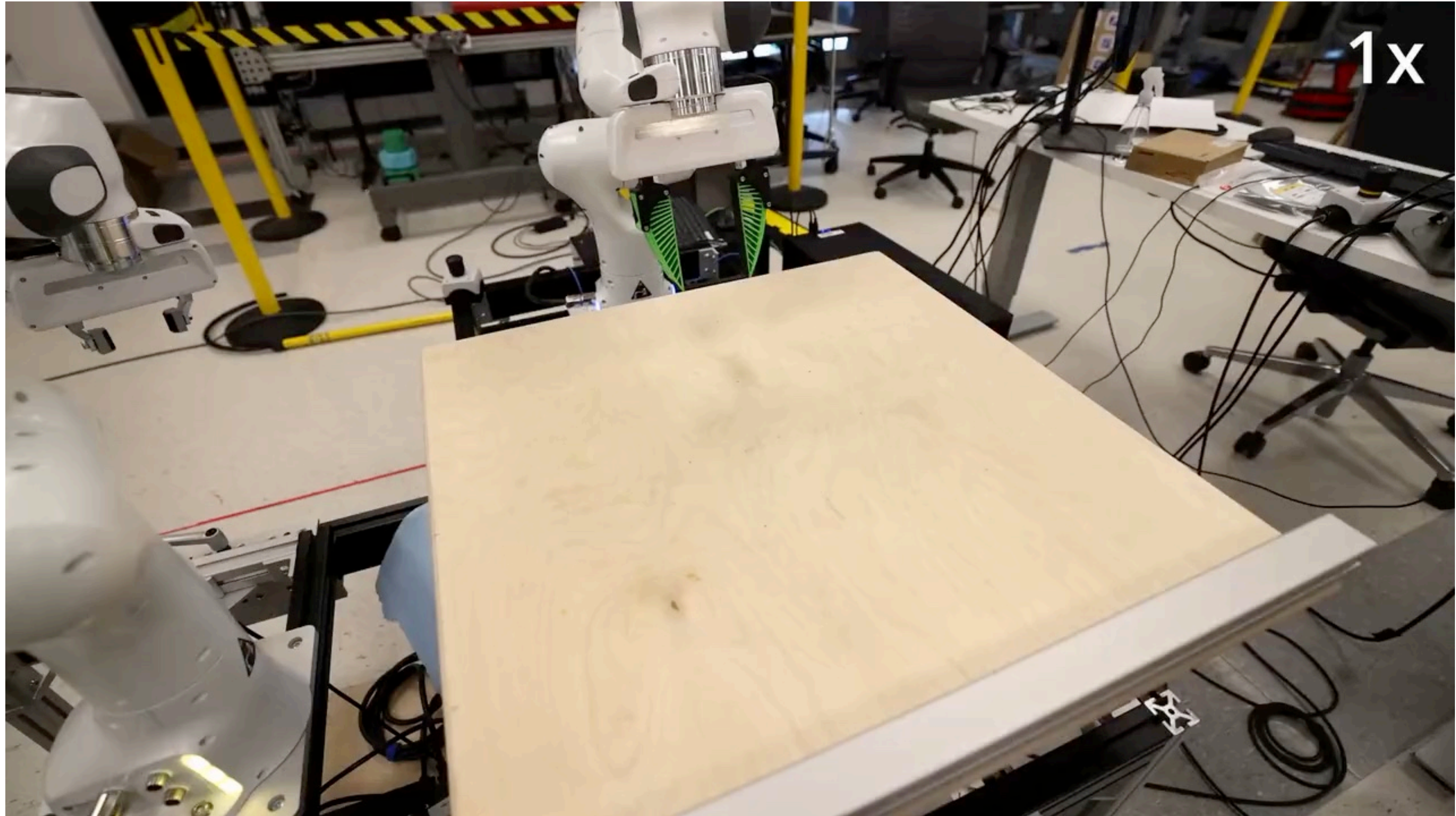
Condition c

0.0

Seed

24

Diffusion policy in action



Practical considerations when training diffusion policies

- Double-check action representation (pre-/post-processing), normalization, gripper actions — well before you train a policy
- Handle zero-actions (remove long swaths of zero-actions); and end-of-episode chunks (e.g., padding / removal as needed)
- Image augmentation helps by making the visual features robust to small changes
- How to determine which checkpoint to select?
 - Train/val loss is often a poor indicator
 - Need closed-loop rollouts, and log metrics other than success (smoothness, retrying behaviors, nature of failures, etc.)

SO-101 + LeRobot defaults

- U-Net, DDPM sampler, 100k training steps, batch size 8
- Action space: absolute joint positions; gripper normalized to [0, 100] (closed/open)
- Action chunk size / horizon -> 16 actions (i.e., ~0.53 seconds, given 30 Hz control frequency)
- By default, 8 of these actions are executed, before we begin executing a new chunk
- Thought exercise(s):
 - What if we execute just 1 action from the chunk?
 - What if we execute the entire action chunk before re-querying?

(More) Administrvia

(More) Administritivia

- Assignment plan, submission process, demo day(s)!
- Most of you have already (hopefully) finished assignment 0 (build and calibrate your SO-101)!
- (soft) deadline tomorrow (9 October) 11.59 pm EDT for assignment 0
 - hard deadline Monday - 12 October 11.59 pm EDT
- Assignments 1, 2, 3 will be released early next week (but we'll walk through the general expectations / deliverables in the next few minutes)

Assignment 1: Modular (sense-plan-act) systems for manipulation

- The course staff will provide a set of cubes and trays (will give out in class next week — please ensure every team has one set)
- Design a modular (perception-planning-control) approach to detect and pick up the cube, and place it on the tray
- In-class demos on Thursday, October 29
- Award for best-performing method (best => fastest + successful)

Assignment 2: Single-task policy learning

- In-class demo on November 12
- Two tasks:
 - cube picking (similar to assignment 1), to be done offline (only video logs + data needed; no in-class demo for this task)
 - Second task <to-be-announced-next-week> — this is the one we'll have an in-class demo for!
- Challenges / considerations:
 - You'll train with data mostly curated outside class, but will be required to demo the task in-class (environment, camera positioning, etc. will change)
 - How do we compensate for it? (Tweak the robot environment design? Collect DAgger data? Something else?)
- Awards for most robust policy and least-strings-attached policy

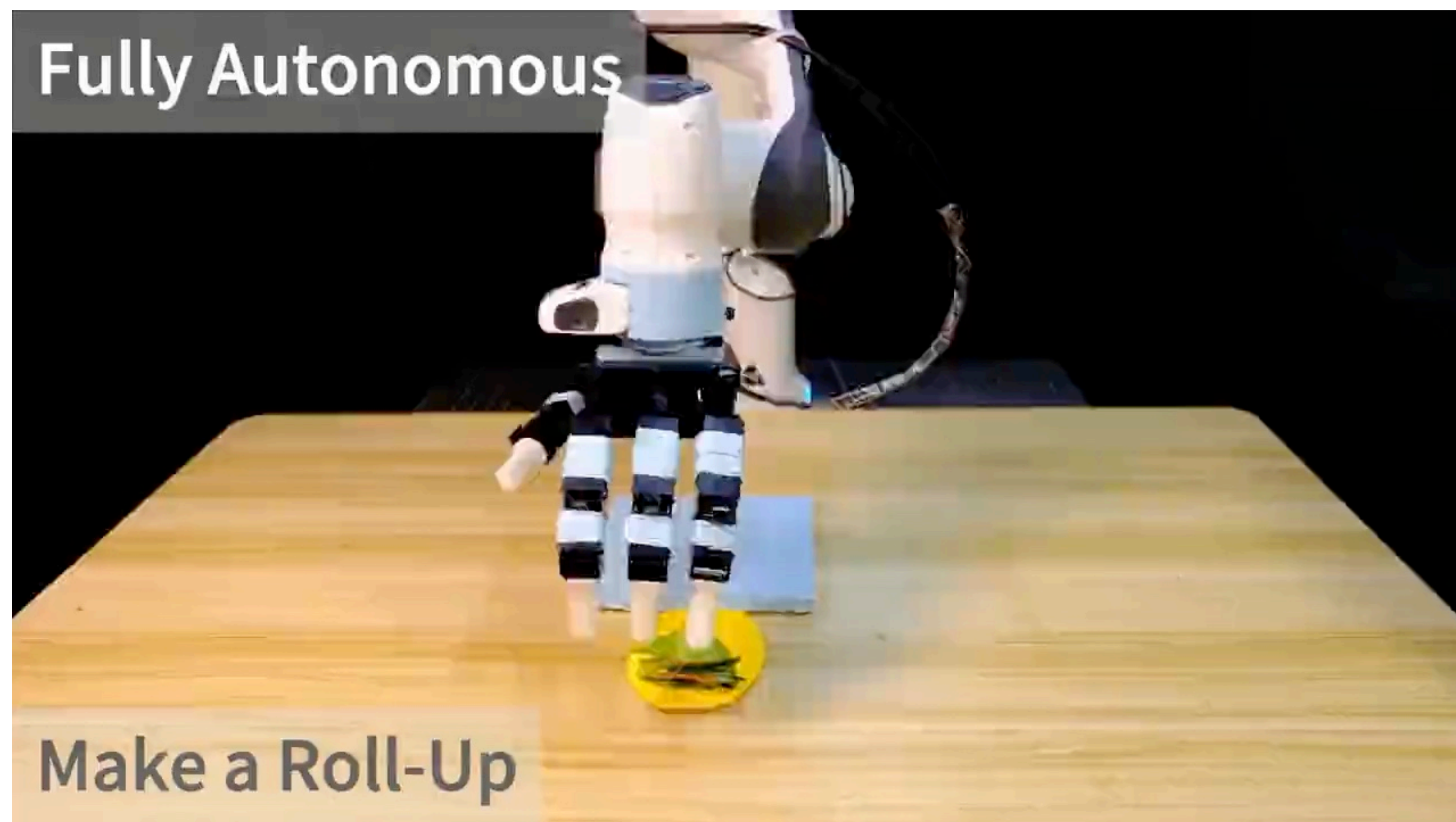
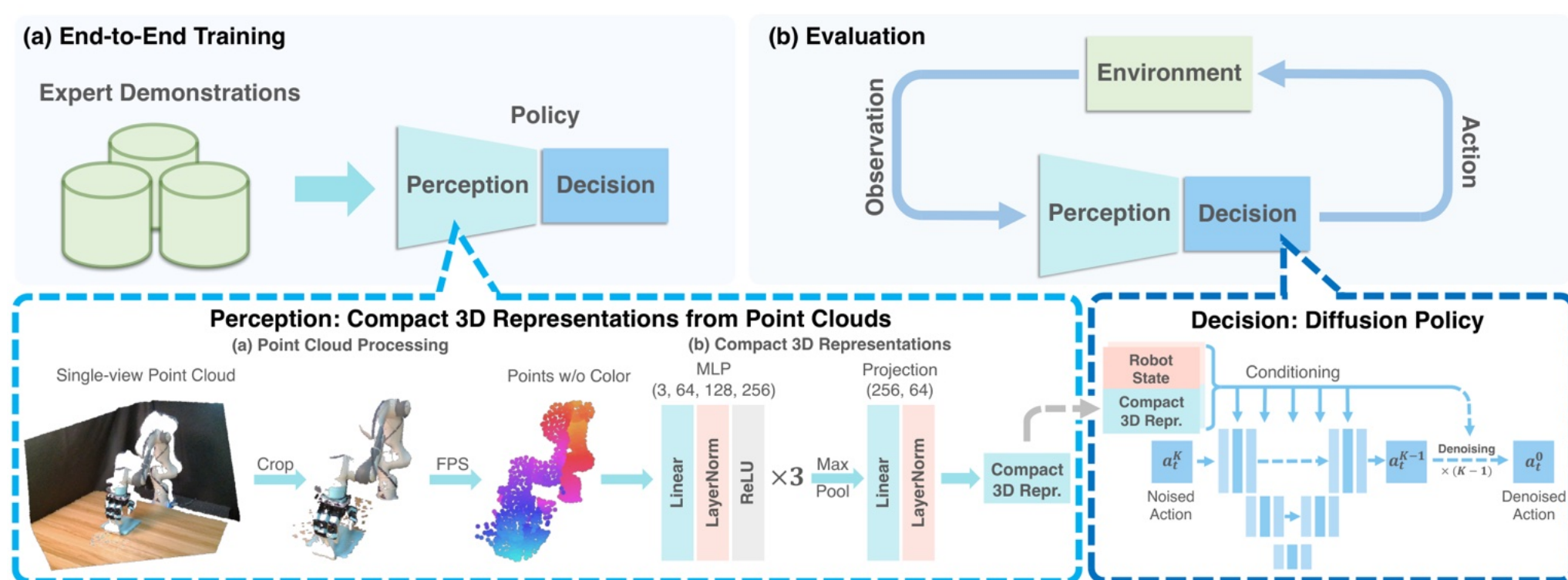
Assignment 3 (mini project): Multi-task policies

- Produce a multi-task policy learning demo and present it in-class on Dec 10
- Panel of faculty judges will provide feedback and nominate projects for awards
- Small hardware modifications permitted — but only with prior authorization from course staff (e.g., if a different gripper is needed)
- We adopt a very generous definition of *multi-task* — if unsure, please check with the CAs or me to ensure your work qualifies

Generative policies (some useful variants)

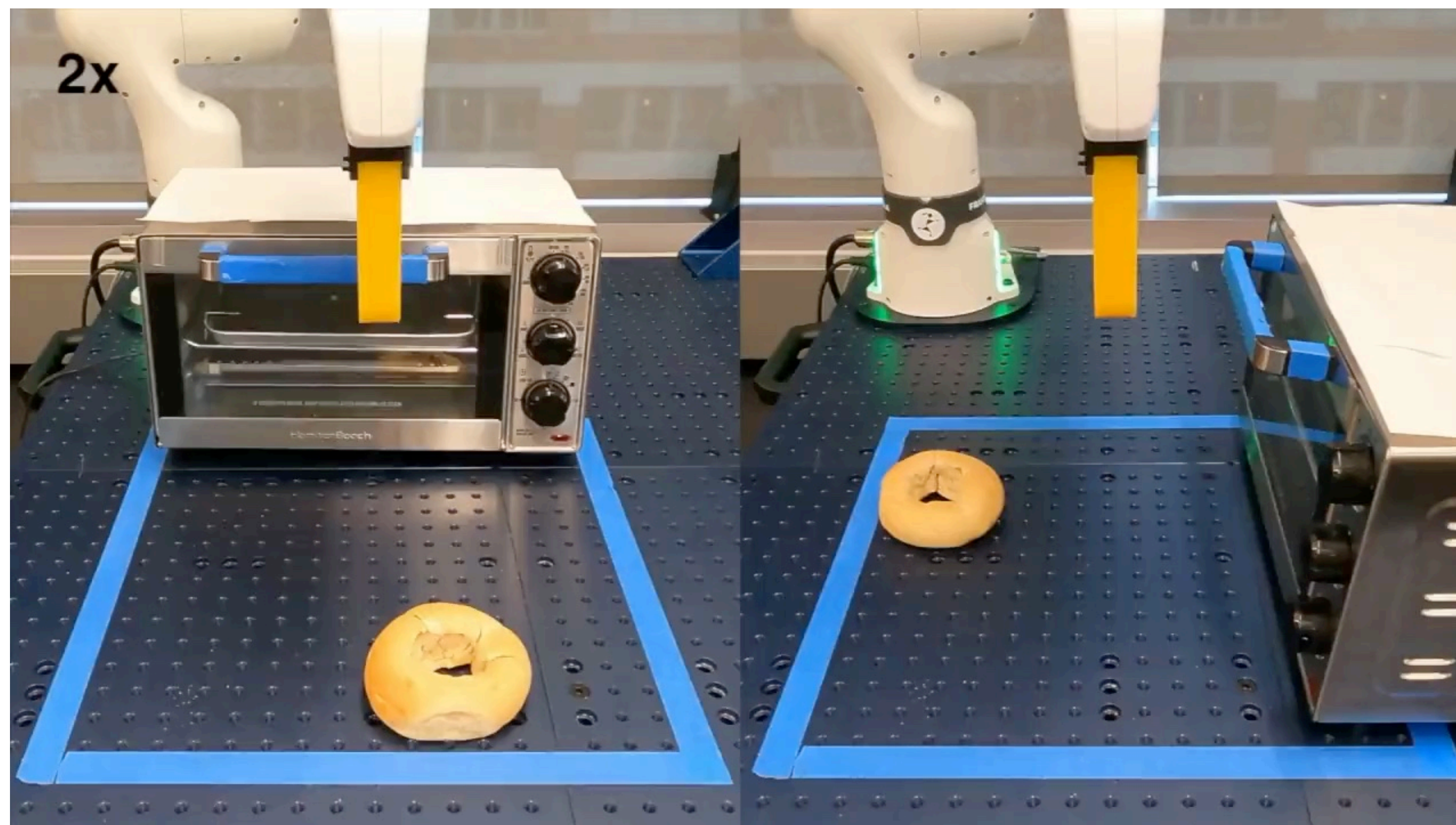
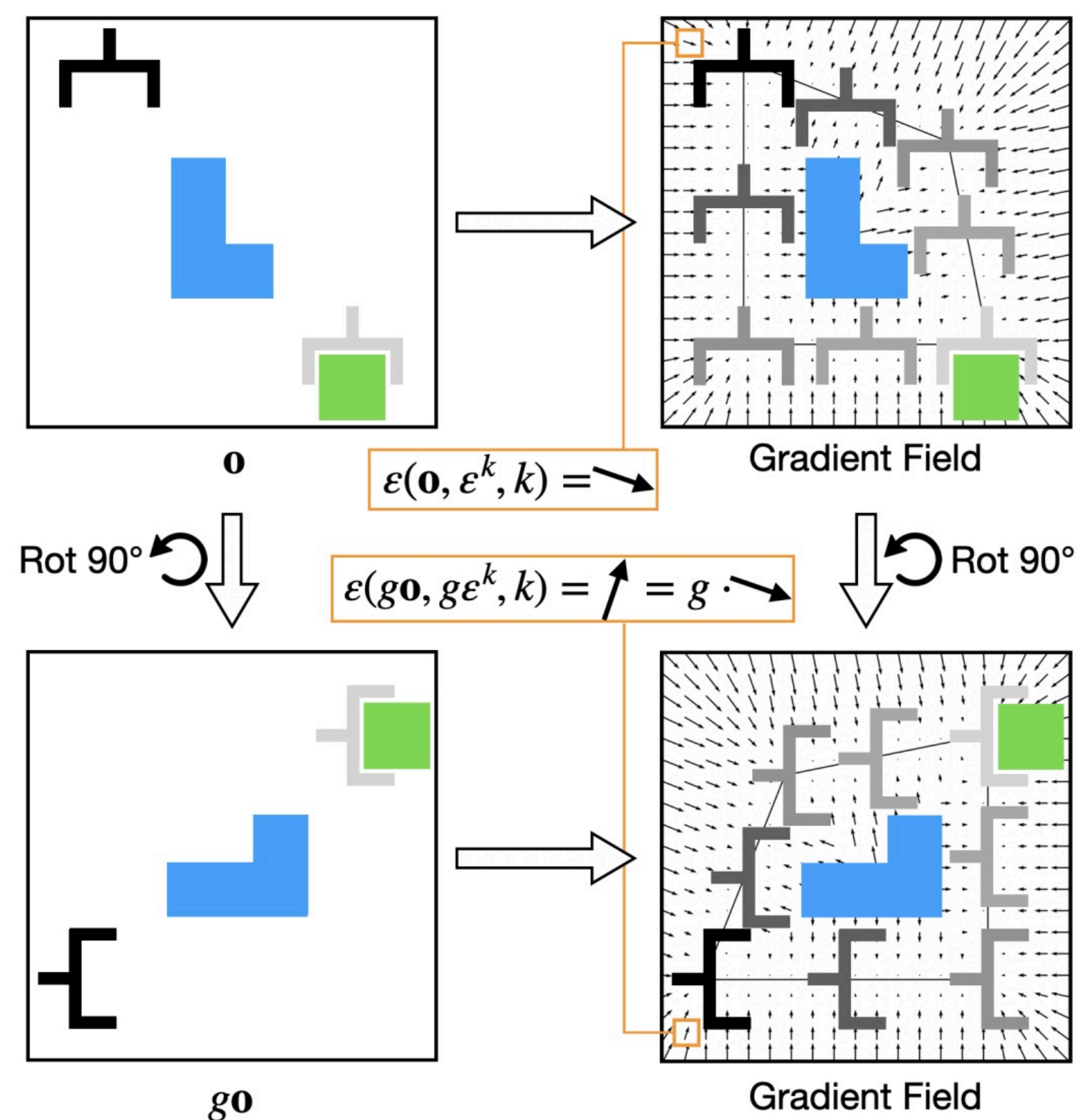
3D Diffusion Policy (DP3)

- DP3 conditions action generation on a compact point-cloud representation
- A Pointcloud encoder (PointNet++) is used as the observation encoder, opposed to vision encoders (ResNet/UNet/ViT) used in vanilla diffusion policy
- Conditions on 3D geometry, so performance is robust to 2D appearance changes like color / texture



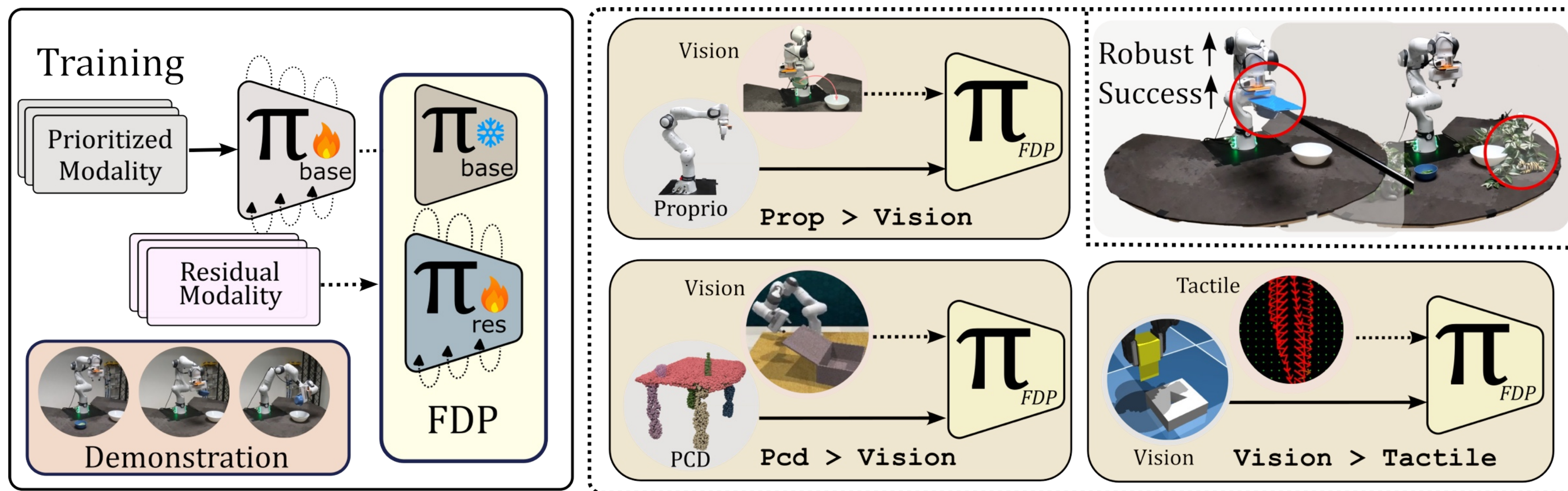
Equivariant Diffusion Policy (EquiDiff)

- Leverage domain symmetries for sample efficiency and better generalization
- If both the state and noise vector are rotated, the policy should adapt (and needn't have to be retrained with those augmentations)

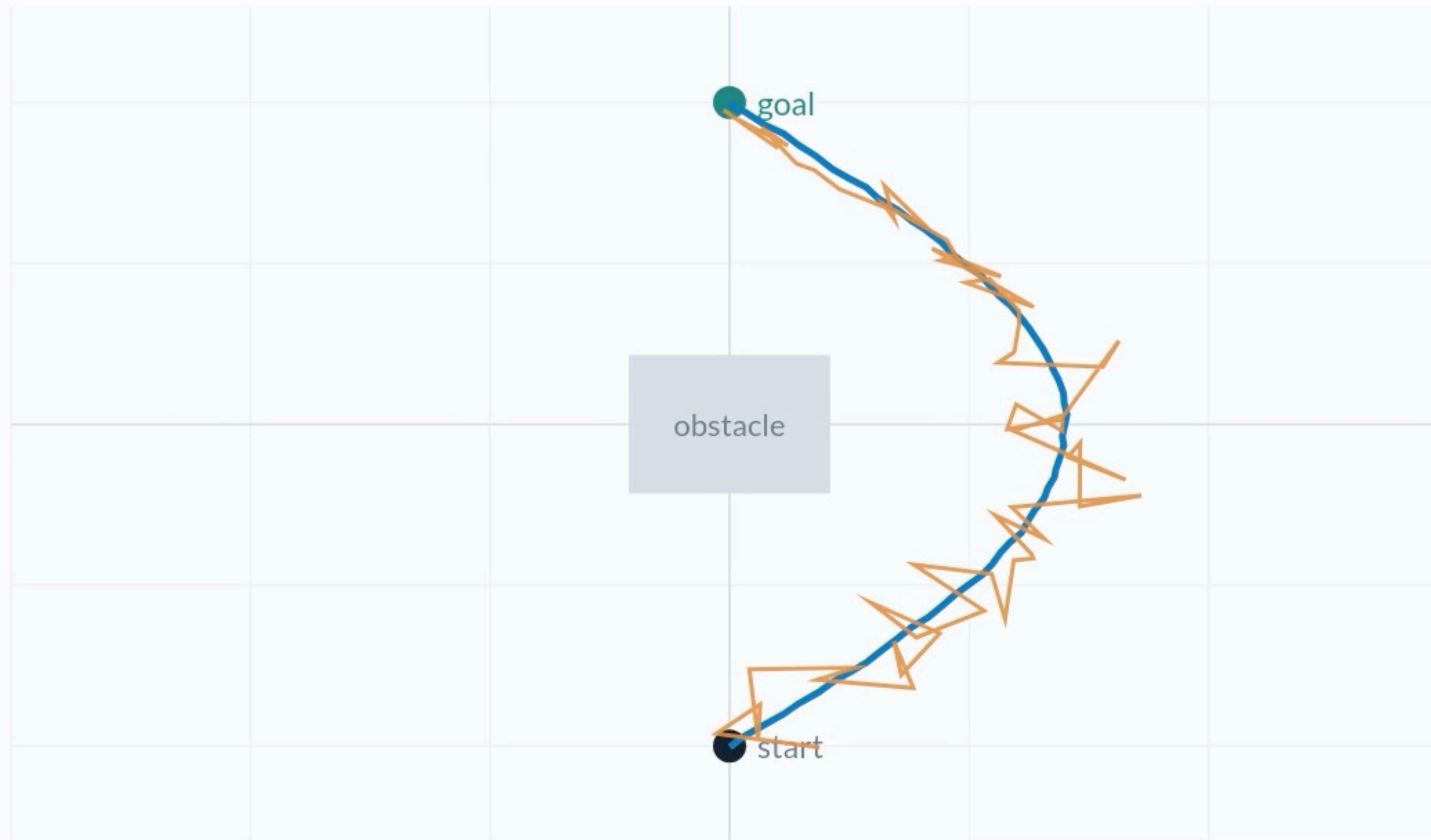


Factorizing Diffusion Policies: observation modalities

- Each modality (robot pose, vision, touch) may contribute different kinds of information
- Factorized diffusion policy controls the contribution of each individual modality by decomposing into prioritized modalities and residual pathways



Which parts of an imperfect demonstration are useful?



Reference motion

Locally imperfect motion

NOISE-DEPENDENT DATA USE

A demonstration can preserve the global route while containing poor local motion. Noise can make those local differences harder to distinguish.

Use data where its useful structure agrees

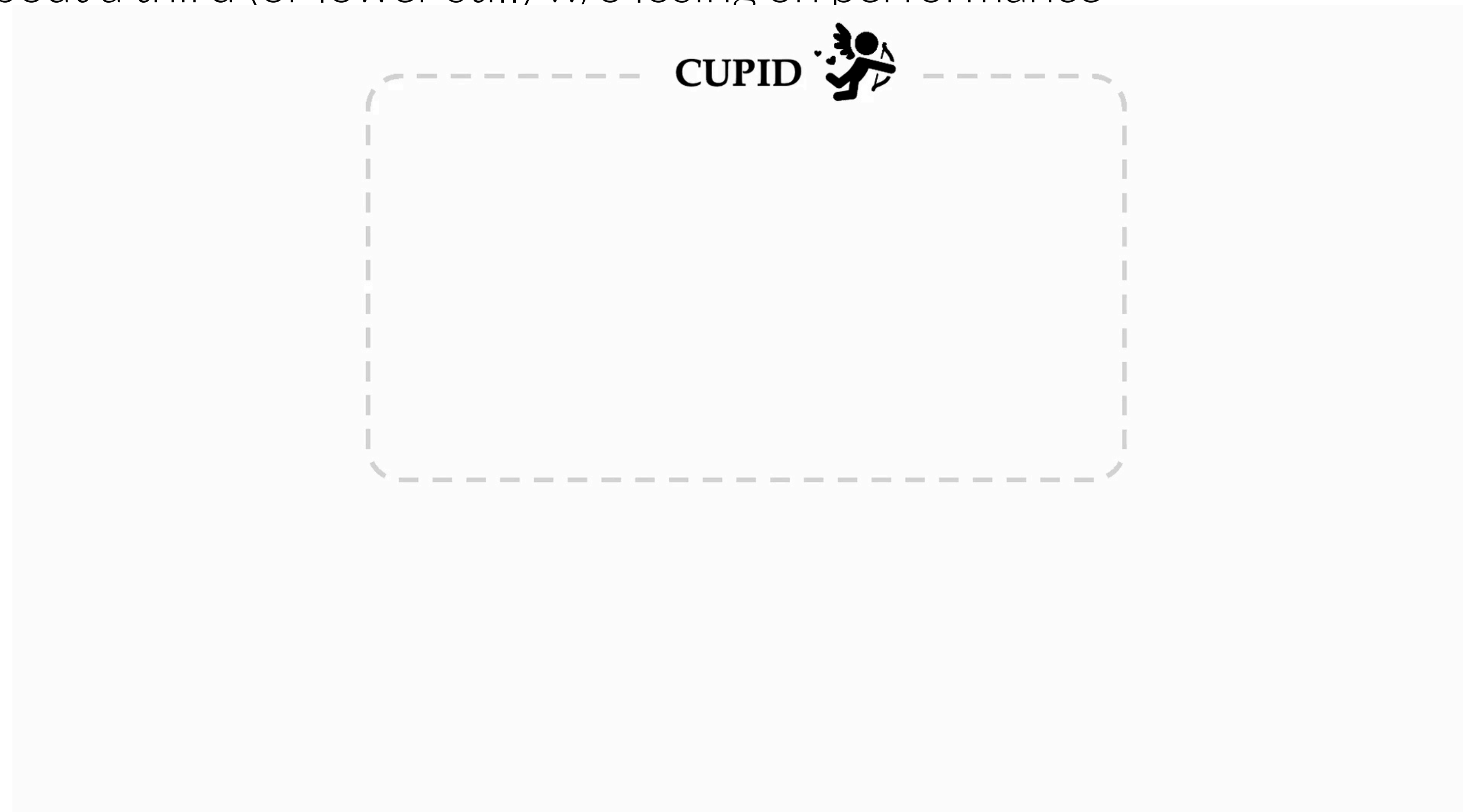
Ambient also considers useful local primitives in globally incorrect behavior at low noise. This drawing illustrates the intuition, not an Ambient training experiment.

Illustrative noise level

0.01

CUPID: Curating data for generative policies

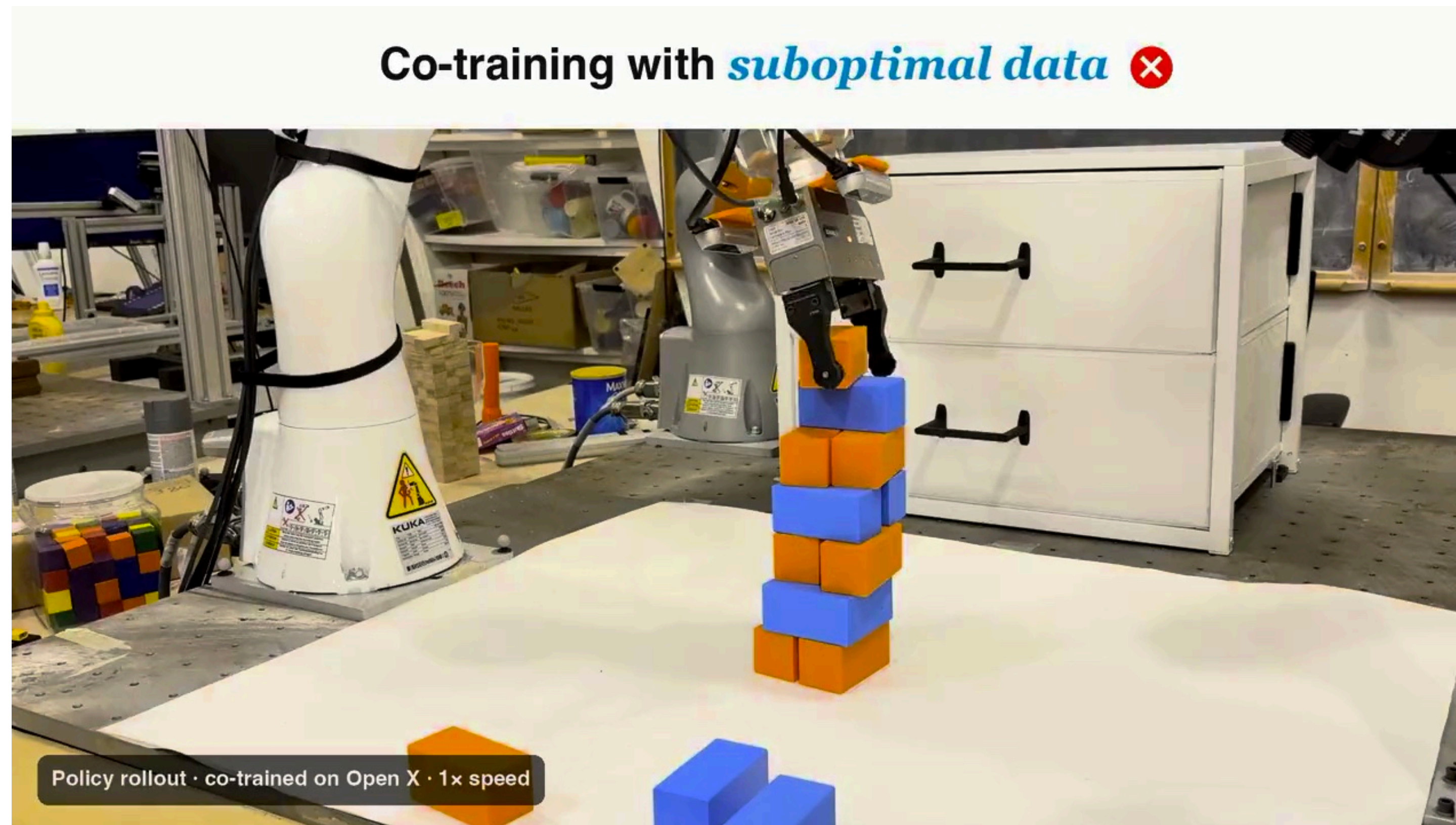
- Tackles the data volume-vs-quality question. Do we need more demos? Or fewer, but higher-*quality* demos
- Uses *influence functions* to estimate causal impact of each demo on policy perf
- Can reduce data to about a third (or lower still) w/o losing on performance*



* This presumes vanilla diffusion policies. The next work we'll look at proposes an alternate way of harnessing all demos (incl. suboptimal ones)

Ambient Diffusion Policy:

- Suboptimal data can still be useful (in at least two regimes): helps learn high-level plans, or low-level consistent motion
- Simple trick / update to diffusion data sampler suffices to implement this change



Controllability of generative (esp. diffusion) policies

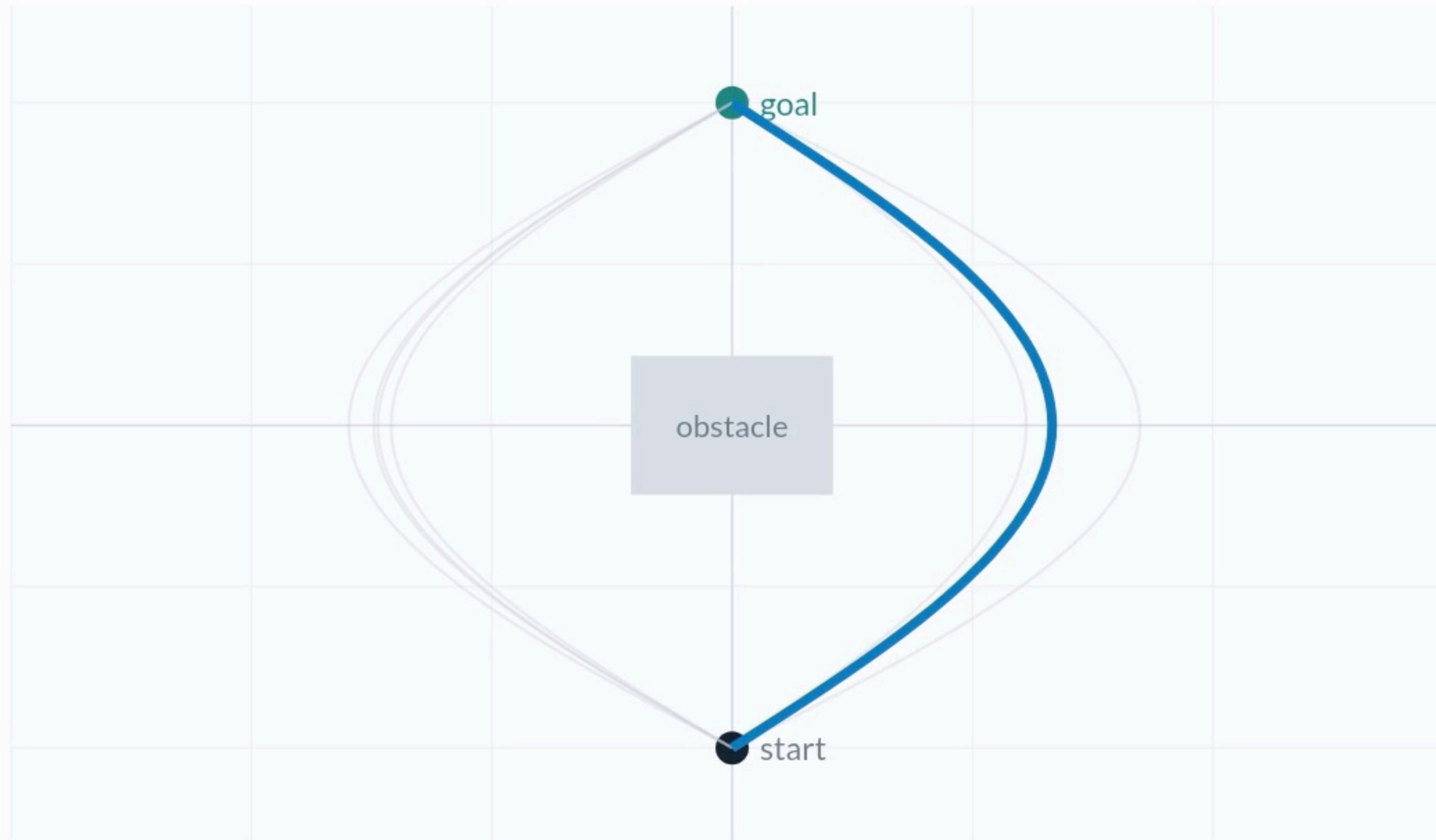
Steering, Composition, Noise vector selection (and downstream impacts)

The diffusion policy as an API

$$a = f_{\theta}(O, z)$$

- Conditioned on observations (images, robot poses, etc.) O and noise vector z , the diffusion model f predicts actions a
- We can, therefore, control a diffusion model by interfering with / tweaking f (via its params θ , observations O , or noise z)
- When we tweak parameters θ , this is commonly referred to as fine-tuning
- When we tweak z , this is commonly referred to as steering
- Other conditioning inputs (such as O) will change over the course of a single policy rollout (as newer images are observed, e.g.)
- We can also *compose* multiple policies ($f, g, \dots$) via simple mathematical/programmatic manipulation

One policy, several noise inputs



CHANGE ONLY THE NOISE INPUT

Different initial noises select different outputs from the same learned policy.

$$A = F\theta(O, z^*)$$

This is a seed-sensitivity illustration. Golden Ticket selects z^* using rollout rewards and a search budget. A visually appealing path is not evidence of task success.

Sampling seed

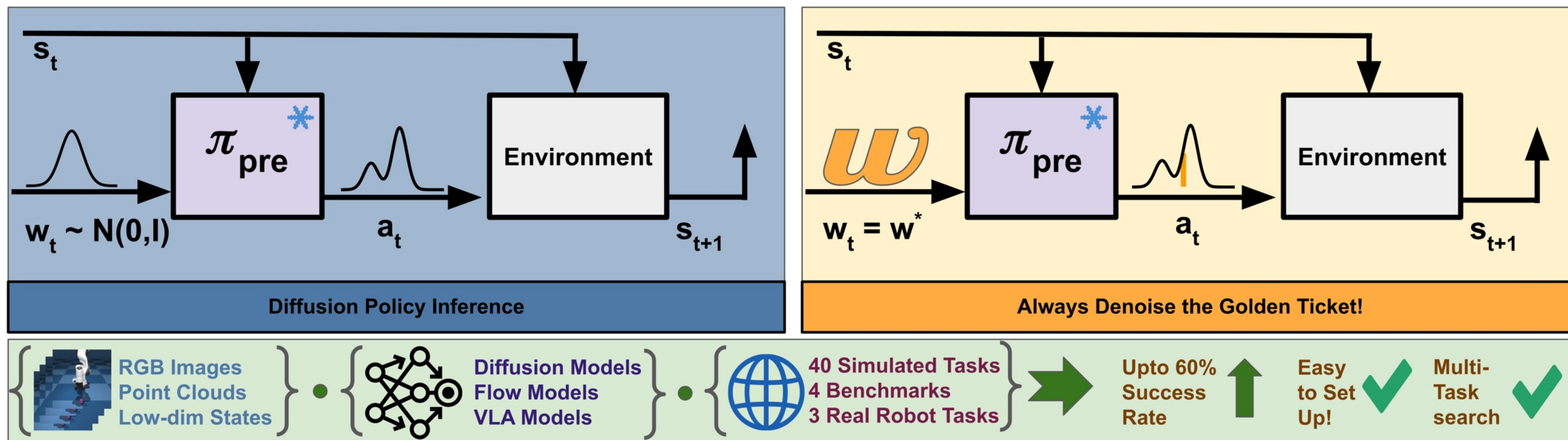
17

Condition c

0.0

Golden tickets: noise vectors that are highly performant

- Sample a few different noise vectors
- For each noise vectors, roll out a policy with that noise vector and compute avg. reward (How? Assumes a reward model is available)
- Keep track of the best noise vector. That's the *golden ticket*



Golden ticket: why/how/does it work?

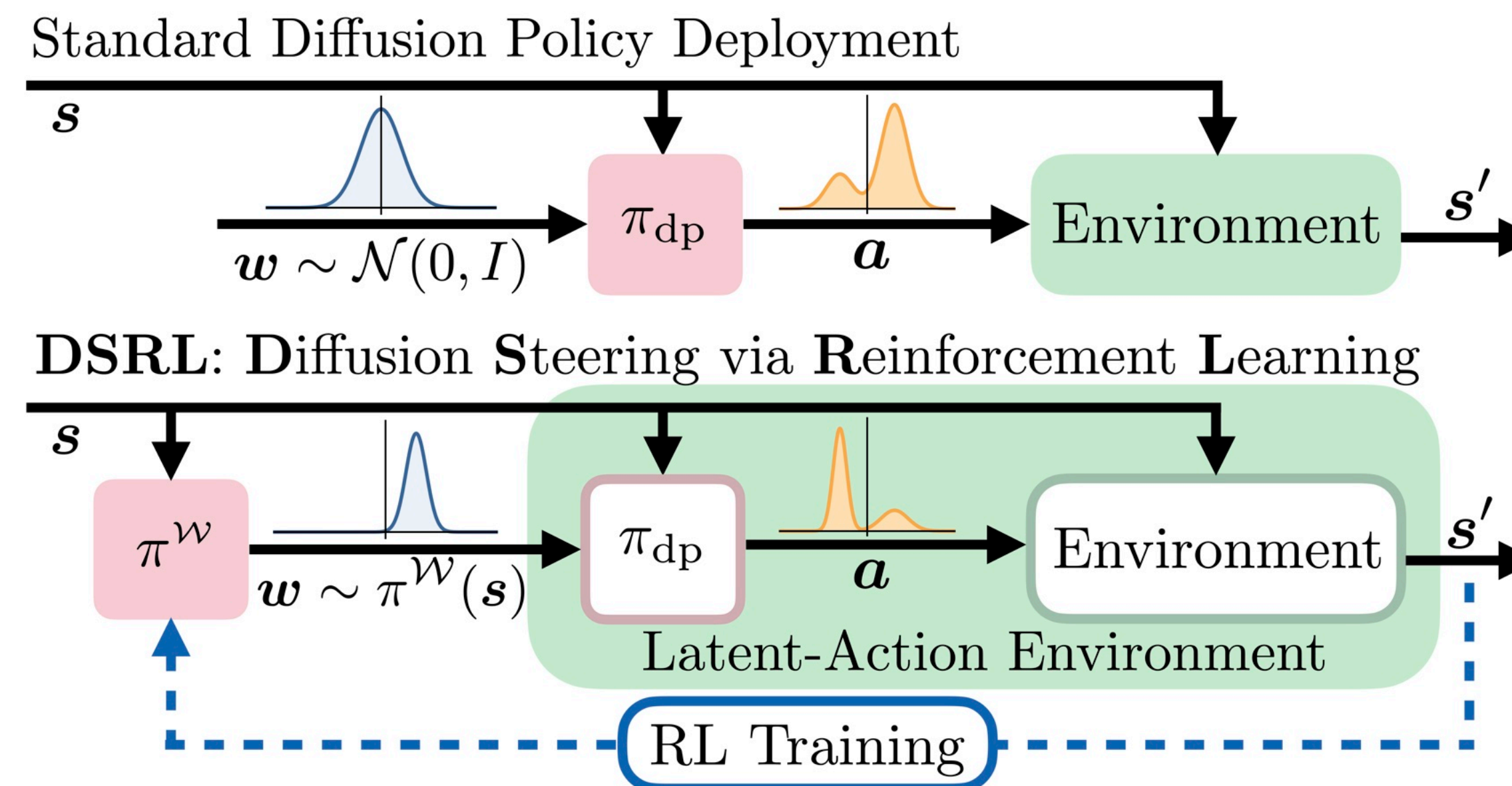
Algorithm 1 Random Search for Optimizing Initial Noise

```
1: Input: Pretrained policy  $\pi$ , set of environments  $E$ , reward  
   function  $R$ , number of tickets  $n$   
2: Output: Optimized initial noise vector  $w^*$   
  
3: Sample  $n$  initial noise vectors  $\{w_i\}_{i=1}^n$ ,  $w_i \sim \mathcal{N}(0, I)$   
4: Initialize best reward  $\bar{R}_{\text{best}} \leftarrow -\infty$   
5: for  $i = 1$  to  $n$  do  
6:   Initialize cumulative reward  $S_i \leftarrow 0$   
7:   for all  $e \in E$  do  
8:     Roll out  $\pi$  in environment  $e$  with fixed noise  $w_i$   
9:     Obtain episode return  $R_e$   
10:     $S_i \leftarrow S_i + R_e$   
11:   Compute average episodic returns  $\bar{R}_i \leftarrow \frac{1}{|E|} S_i$   
12:   if  $\bar{R}_i > \bar{R}_{\text{best}}$  then  
13:      $\bar{R}_{\text{best}} \leftarrow \bar{R}_i$   
14:      $w^* \leftarrow w_i$   
15: return  $w^*$ 
```

- “Lottery ticket hypothesis” (specific randomly initialized Gaussian noise vectors contain special noise blocks that decode to better signal independently)
- Fixed noise input can select useful behavior supported by the policy (but can’t synthesize / extrapolate behaviors that aren’t captured by the policy)

DSRL: Diffusion steering via reinforcement learning

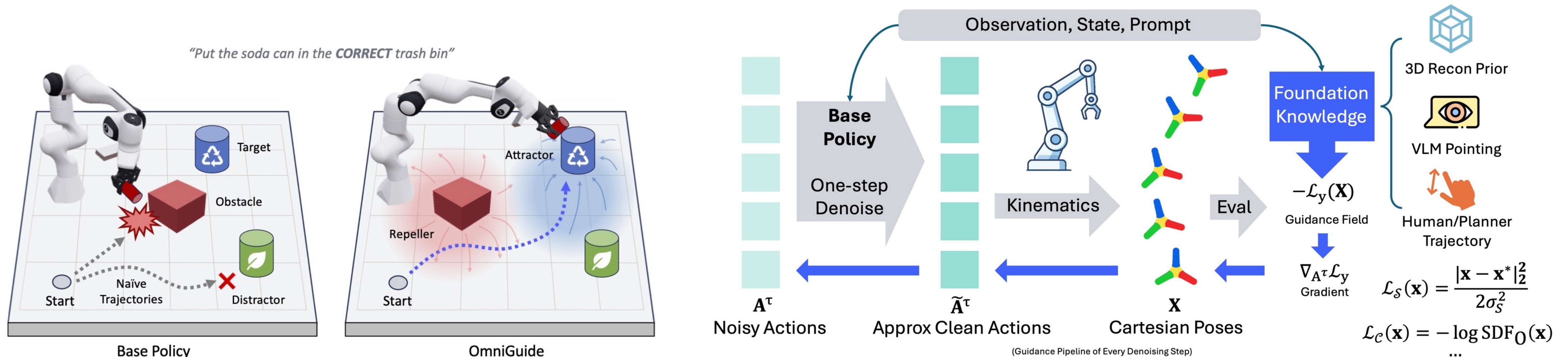
- Diffusion policies can be fine-tuned, but what if
 - we don't have too many finetuning examples, or
 - we don't want to (or are unable to) backprop through large models (e.g., VLAs) but would like a lightweight fine-tuning approach
- DSRL trains a small MLP that predicts noise vectors, instead of fine-tuning the diffusion / flow-matching policy directly



Steering policies via *guidance*

- Learned policies provide a *prior* over plausible behavior
- We can encode costs that denote additional preferences
- Cost can be used during the denoising process (so long as it is differentiable)

$$p^*(a \mid o) \propto p_\theta(a \mid o) \exp[-\lambda C(a, o)] \quad \nabla_a \log p^* = \nabla_a \log p_\theta - \lambda \nabla_a C$$



Steering policies via *guidance*

- In practice, at each denoiser step t ,
 - Run a forward pass through the model to get the noise vector $\epsilon = \epsilon_{\theta}(a_t, t, o)$
 - Compute a one-step estimate of the denoised action $\hat{a}_0 = \frac{a_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$
 - Evaluate C on this one-step estimate (C is the cost), doing a backward pass through the denoiser if needed
 - Add to the predicted noise $\epsilon \leftarrow \epsilon + \lambda_t \sqrt{1 - \bar{\alpha}_t} \nabla C$

Choosing where generated actions should go



AN EXACT TOY PRODUCT

A soft preference changes the relative probability of the two modes and shifts samples within each mode.

$$p^*(A) \propto p(A) \exp(-\lambda C(A))$$

These samples come from an analytically computed product. Practical diffusion guidance approximates the corresponding noisy-time conditioning. It is not a guaranteed constraint solver.

Preference strength λ

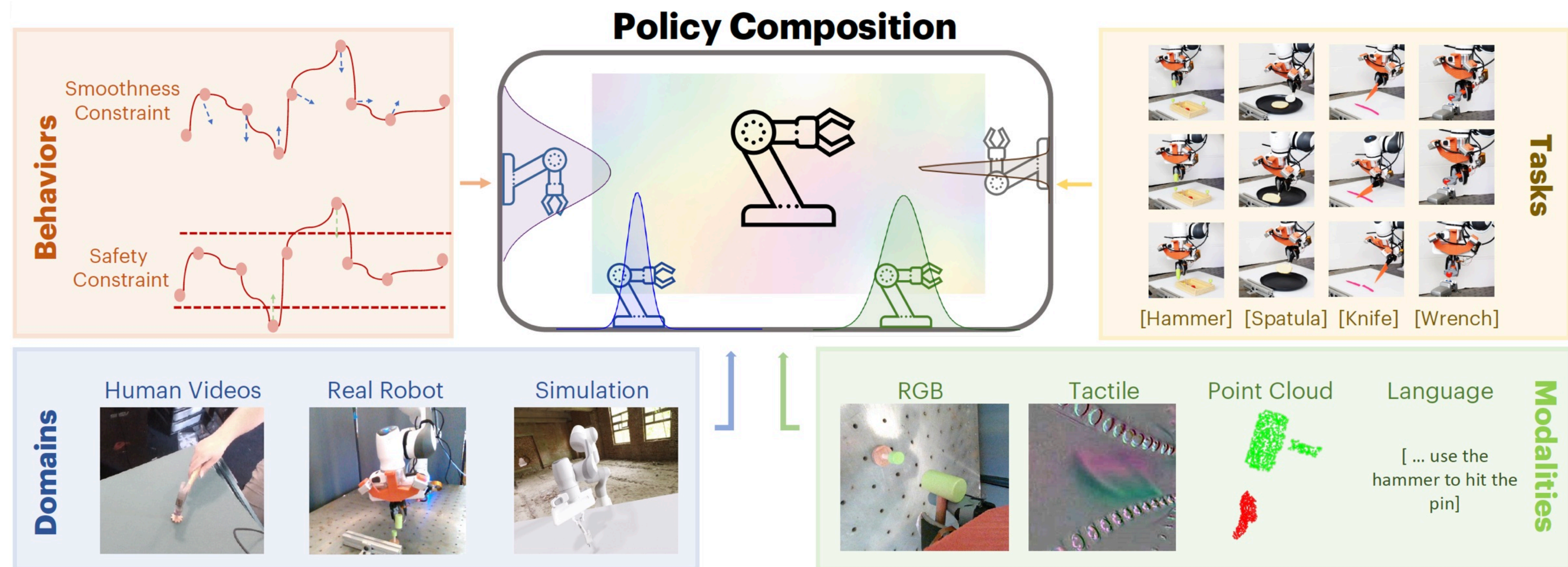
0.00

Right-mode probability

50%

Policy Composition

- At inference time, compose policies by combining gradient predictions (compose noise vectors)
- Can be used to combine tasks/modalities/domains/behaviors
- Only needs a small change to denoising implementation (denoise across multiple models, instead of one)



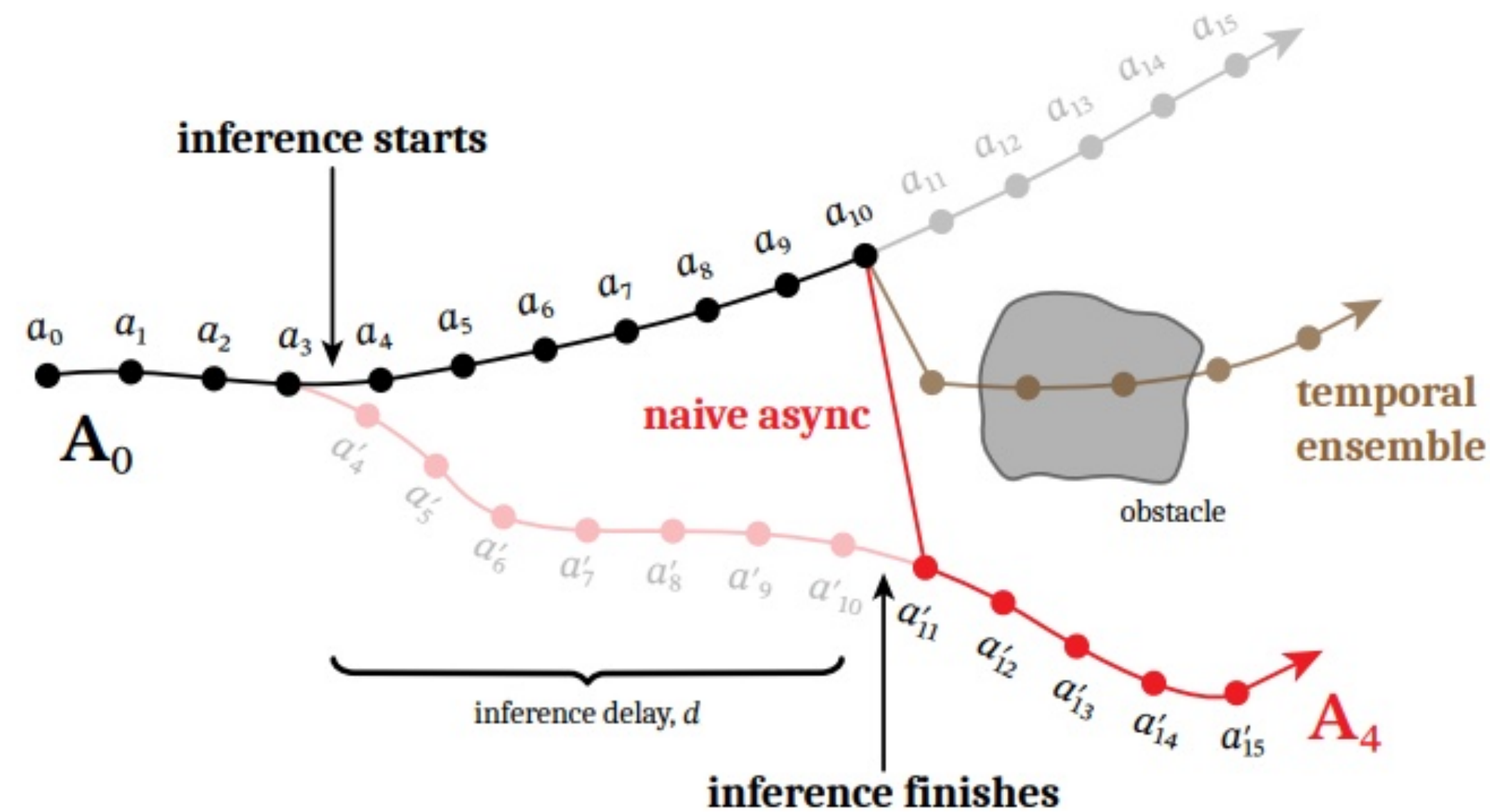
Inference stack for generative policies

Inference with action chunks in general

- (Discussion applies to all action chunking policies, whether or not they're generative)
- Two parameters we have control over: prediction horizon and execution horizon
- In practice we execute only a subset of predicted actions. This enables reactive behaviors, while ensuring we aren't waiting on the policy to predict newer actions
- This matters because, in practice, policies are quite slow ($O(10)$ - $O(100)$ milliseconds) while controllers are fast (30-1000 Hz)
- However, these processes (observation, prediction, execution) run asynchronously
- This means the policy is acting on older observations, predicting actions that are committed to w/o factoring in latest state changes
- Action chunking transformers used temporal ensembling to mitigate some of these effects
- (Subsequent discussion on the next slide and thereafter specifically applies to generative policies)

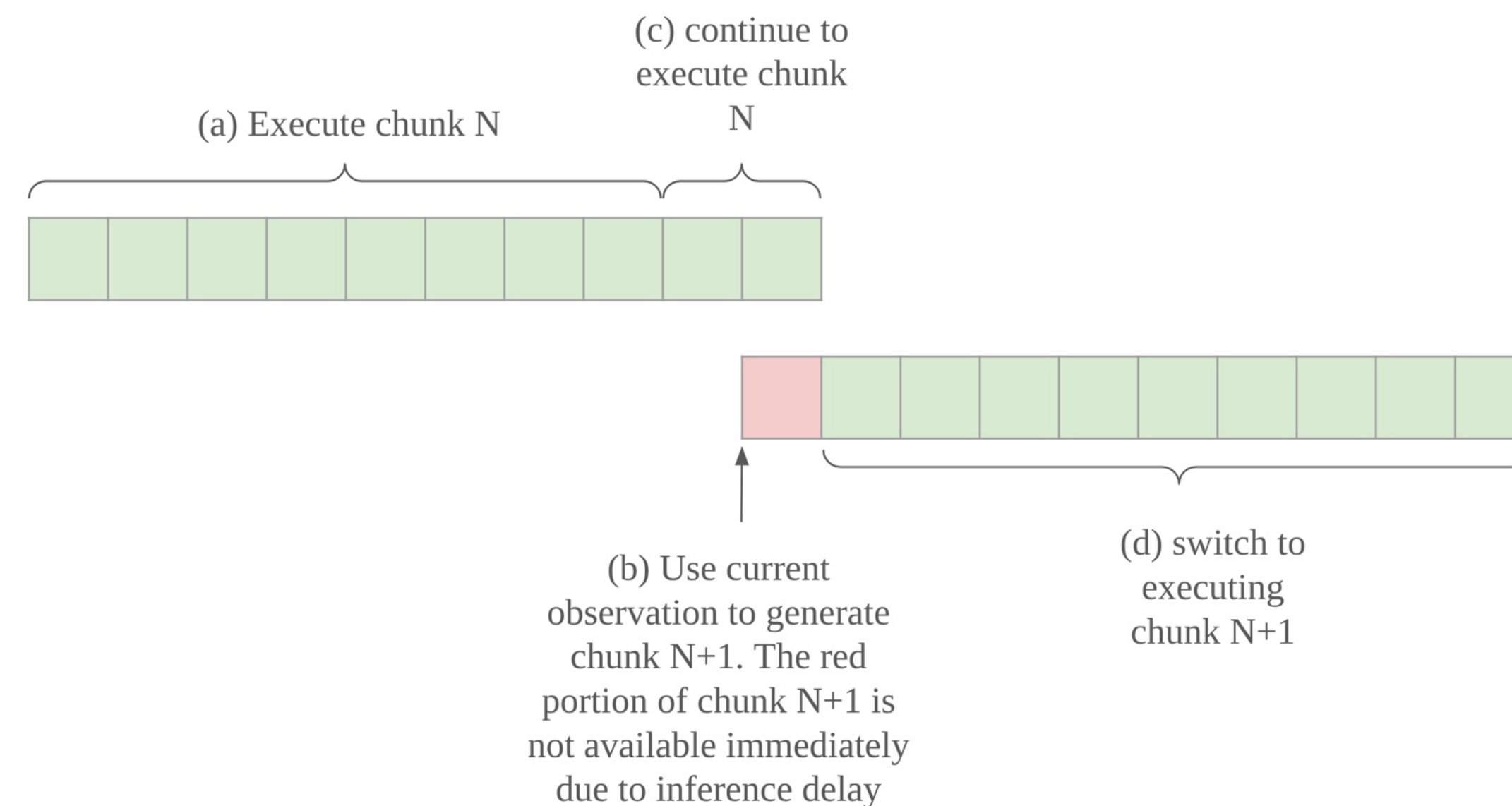
(Naive) Approach 1: Asynchronous switching

- (Simplest to implement, but doesn't work well in practice)
- Idea: switch as soon as a new action chunk is available
- Little control over what *mode* the new chunk latched onto
- Unsmooth / discontinuous behavior qualitatively (even when mode switching doesn't occur)



Approach 2: Compute the next chunk before it is needed

- i.e., compute next chunk before the current action chunk finishes
- this can be necessary to prevent the policy from being OOD (e.g., if all your policy has seen was velocity data / pose history, stop-starting results in OOD inputs)
- Real-time action chunking: *inpaint* each subsequent chunk with a portion from the end of next chunk
- Smooth-as-butter policies: additionally add observation as a cue during real-time action chunking



Real-time action chunking

- The start of a new chunk is constrained by commands that'll be executed while inference is ongoing
- Overlapping region (a hyperparameter) enables continuity wrt previous chunk
- Further out into the future, the constraint is relaxed — to enable the policy to respond to changed observations
- Note: this optimizes for smoothness, not necessarily reactivity
- Additional readings for those interested in a deeper dive:
 - Real-time action chunking (Physical intelligence)
 - Smooth-as-butter policies (Alexander Soare's blog)
 - Training time action conditioning (Black et al. 2025)
 - Bidirectional decoding (Liu et al. ICLR 2025)
 - Streaming diffusion policy (Hoeg et al. ICRA 2025)
 - Learning native continuation for action chunking flow policies (Liu et al. 2026)

Generative policies: Key takeaways

- Adept at handling multimodality
- Training and inference are (slightly) more involved compared to action-chunking transformers
- Diffusion / flow matching policies can be steered / composed / tuned in various ways (much of which don't require any changes to the model itself)
- Inference-time considerations (prediction-vs-execution horizon, smoothness-vs-responsiveness)

Questions?